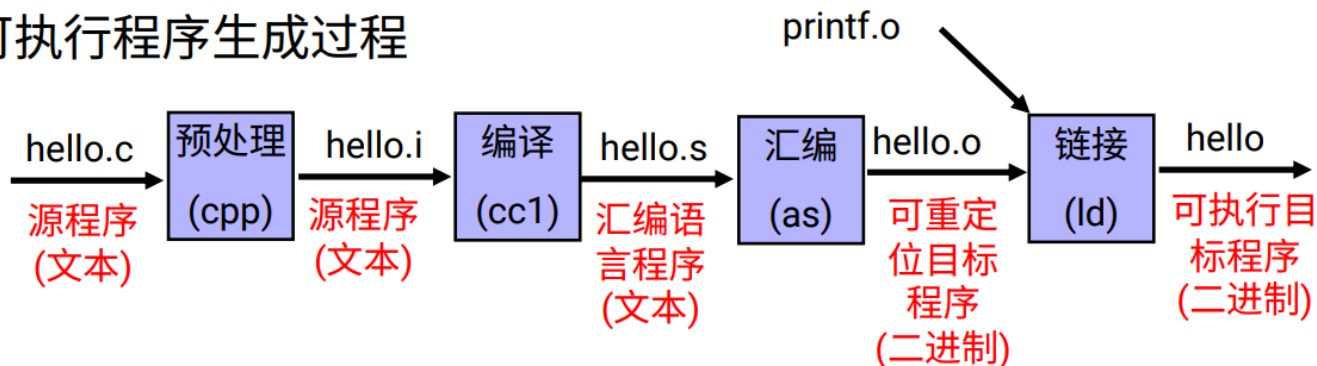


Chapter7 链接

链接（Linking） 是将多个**可重定位目标文件**（如 `.o` 文件）和**库文件**合并生成最终**可执行目标程序**的关键步骤。它的核心作用是**解决模块间的引用关系**，使程序能正确运行

可执行程序生成过程



使用链接器的理由：

1. 模块化：程序可以编写为一个较小的源文件的集合，而不是一个整体的大文件。同时可以构建公共函数库以供代码复用。
2. 效率：时间上不需要重新编译其他源文件，空间上可以将公共函数聚合为单个文件

链接过程有两个部分：符号解析和重定位

符号解析：确定符号的地址或位置

符号：程序中有定义和引用的符号（如全局变量和函数），局注意部变量和参数等经编译器处理后不再是符号了。

符号的定义由汇编器存储在目标文件中的符号表中。符号表：一个结构型数组。

链接器将每个符号引用正好与一个符号定义关联起来

重定位：将程序中的地址引用修正为实际运行时的地址

将多个代码段与数据段分别合并为一个单独的代码段和数据段

计算每个定义的符号在虚拟地址空间中的绝对地址

将可执行文件中符号引用处的地址修改为重定位后的地址信息

目标文件格式

标准的目标文件格式

- Linux&Unix：ELF
 - Windows：PE
- 三种目标文件（都是ELF二进制）
- 可重定位目标文件(`.o`)
- 包含与其他可重定位目标文件相结合的代码和数据，以形成可执行的目标文件。`.o`文件是由源文件(`.c`)生成，代码和数据地址从0开始

- 可执行目标文件(`a.out`, `.exe`)
包含可以直接复制到内存并执行的代码和数据
代码和数据地址为虚拟地址空间中的地址
- 共享目标文件(`.so`)
特殊类型的可重定位目标文件。在其他的程序加载时或运行时，它可以被动态加载到内存，
并被动态链接。在Windows中称为动态链接库(.DLL)
使用 `readelf` 工具来查看或分析指定的文件

ELF目标文件格式(`cont.`)

ELF头
段头表(可执行文件才有)
.text节
.rodata节
.data节
.bss节
.symtab节
.rel.txt节
.rel.data节
.debug节
节头表

- **ELF头**： 字大小、字节顺序、文件类型(`.o`, `exec`, `.so`), 机器类型 (如IA-32), **节头表的偏移**、节头表的表项大小以及表项个数等等
- **段头表/程序头表 (可执行文件)**： 页面大小，虚拟地址内存段(节)，段大小

- **节头表**：每个节的节名、偏移量和大小
- `.text`：代码
- `.rodata`：只读数据（printf、跳转这些）
- `.data`：可读写数据，比如已初始化全局变量
- `.bss`：未初始化全局变量
- `.symtab`：符号表、函数和静态变量名、节名称和位置
- `.rel.text`：可重定位代码。`.text` 节的可重定位信息、在可执行文件中需要修改的指令地址 需修改的指令
- `.rel.data`：可重定位数据。`.data` 节的可重定位信息、在合并后的可执行文件中需要修改的指针数据的地址
- `.debug`：符号调试信息
- `.strtab`：包含 `symtab` 和 `debug` 节中的符号及节名

节头表

节头表是ELF可重定位目标文件中最重要的内容，描述每个 节的节名、在文件中的偏移、大小、访问属性、对齐方式等

\$ readelf -S test.o #打印节头表信息

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	00000000	000034	00005b	00	AX	0	0	4
[2]	.rel.text	REL	00000000	000498	000028	08		9	1	4
[3]	.data	PROGBITS	00000000	000090	00000c	00	WA	0	0	4
[4]	.bss	NOBITS	00000000	00009c	00000c	00	WA	0	0	4
[5]	.rodata	PROGBITS	00000000	00009c	000004	00	A	0	0	1
[6]	.comment	PROGBITS	00000000	0000a0	00002e	00		0	0	1
[7]	.note.GNU-stack	PROGBITS	00000000	0000ce	000000	00		0	0	1
[8]	.shstrtab	STRTAB	00000000	0000ce	000051	00		0	0	1
[9]	.symtab	SYMTAB	00000000	0002d8	000120	10		10	13	4
[10]	.strtab	STRTAB	00000000	0003f8	00009e	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings)

I (info), L (link order), G (group), x (unknown)

O (extra OS processing required) o (OS specific), p (processor specific)

Name：节名

Type：节的类型（无效/保存程序相关信息/符号...）

Addr：虚拟地址 • Off: 在文件中的偏移地址

Size：节在文件中所占长度

ES(Entsize): 节中每个表项的长度, 0表示无固定长度表项

Lk、Inf: 用于与链接相关的节 (如 .rel.text节、.rel.data节 、.symtab节 等)

AI(Addralign): 节的对齐要求

没见考过这个, 作业里也没有

符号解析

每个重定位目标模块m都有一个符号表(.symtab节), 它包含定义和引用的符号信息, 有三种不同的符号:

- **全局符号**(non-static): 由该模块定义并能被其他模块引用的符号。如 non-static C 函数和 non-static 的 C 全局变量
- **外部符号**: 由该模块引用的全局符号, 但由其他模块定义
- **本地/局部符号**(static): 由该模块定义和仅由该模块唯一引用的符号, 不能被外部模块引用。如 static C函数和 static 全局变量。存储在栈或寄存器上 (运行时), **不等于非 static 局部变量 (存储在 .bss 或 .data)**。
符号对应于一个函数、一个全局变量或一个静态变量(变量指 static 属性声明的变量)

强弱符号和处理

强弱符号是针对全局符号的概念。程序符号要么是强符号, 要么是弱符号。强符号是**函数和初始化全局变量**, 弱符号是**未初始化的全局变量**。

以下关于程序中链接“符号”的陈述, 错误的是 (B)

A. 赋初值的非静态全局变量是全局强符号

B. 赋初值的静态全局变量是全局强符号

号

C. 未赋初值的非静态全局变量是全局弱符号

D. 未赋初值的静态全局变量是本地

符号

静态全局变量是本地符号而不是全局符号

处理规则

1. 不允许多个同名的强符号, 否则**链接器错误**
2. 若有一个强符号和多个弱符号同名, 引用时都解析为强符号
3. 若有多个弱符号同名, 选择任意一个。可以用 gcc -fno-common 避免
例子

```
int x;  
int y;
```

```
p1(){}


```

```
double x;
p2(){}


```

在p2中写入x可能会覆盖y: x都是弱符号, 随机选择。如果选到double没事。如果选到int, 由于p2使用的是基于x首地址的连续8个字节, 就会导致double x的高4个字节和int y在内存上重叠, 此时写入double x会覆盖y。

```
int x=7;
int y=5;
p1(){}


```

```
double x;
p2(){}


```

在p2中写入x将覆盖y。int x=7是强符号, 会被链接器选择, 出现上面说的重叠情况

符号存放位置

编译时存储在目标文件 .o 的**符号表**, 链接时存储在可执行文件的**全局/静态存储区**。具体有三节/段

- .text : 已编译程序的机器代码, 比如`void fun(){} , printf(), static void fun(){}'
- .data : 已初始化的静态和全局C变量, 比如 int x=1, static int y=2
- .bss : 未初始化的静态和全局C变量, 初始化为0的静态和全局变量

重定位

为什么要重定位?

当汇编器生成一个可重定位目标模块时, 它并不知道数据和代码最终将放在内存中的什么位置, 也不知道这个模块引用的任何外部定义的函数或者全局变量的位置。所以它会生成一个**重定位条目**, 告诉链接器在将目标文件合并成可执行文件时如何修改这个引用。代码的重定位条目放在 .rel.text 中。已初始化数据的重定位条目放在 .rel.data 中。

重定位有**三步**

1. 合并相同的节。所有目标模块中相同的节合并成新节。例如, 所有 .text 节合并作为可执行文件中的 .text 节

2. 对定义符号进行重定位（确定地址）。确定新节中所有定义符号在虚拟地址空间中的地址。例如，为函数确定首地址，进而确定每条指令的地址，从而为变量确定首地址。完成这一步后，每条指令和每个变量的地址都可确定
3. 对引用符号进行重定位（确定地址）。修改 `.text` 节和 `.data` 节中每个符号的引用（地址）。需要用到在 `.rel.data` 和 `.rel.text` 节中保存的**重定位信息**（基本就是上面的重定位条目）

可重定位条目

可重定位条目的结构：使用 `Elf64_Rela` 结构体表示可重定位条目，包含以下字段：

- `offset`：需要被修改的引用的节内偏移。
- `type`：重定位类型（32位）。
- `symbol`：符号表索引（32位）。
- `addend`：重定位表达式中的常量部分（调整偏移）

```
typedef struct {
    long offset;      /* 需要被修改的引用的节内偏移 */
    long type:32,     /* 重定位类型（如PC相对或绝对地址） */
        symbol:32;   /* 符号表中的索引（指向哪个符号） */
    long addend;      /* 重定位表达式中的常量偏移 */
} Elf64_Rela;
```

相当于告诉链接器：在文件的某个位置（`offset`），有一个对符号（`symbol`）的引用，需要按照某种方式（`type`）调整它的值，并可能加上一个常量（`addend`）

ELF 重定位类型

1. `PC32`：PC 相对地址，PC 值通常是下一条指令在内存中的地址。
2. `32`：绝对地址。

调整偏移

方式由重定位类型决定。被修改的引用指向的符号信息存放在 `.rel.data`（可重定位数据）或 `.rel.text`（可重定位代码）节中

重定位算法

链接器的重定位算法可以概括为以下步骤（伪代码）

```
foreach section s {
    foreach relocation entry r in s {
        refptr = s + r.offset;  // 定位到需要修正的引用位置
```

```

    if (r.type == R_X86_64_PC32) {
        // PC相对地址修正
        refaddr = ADDR(s) + r.offset; // 当前引用的运行时地址
        *refptr = ADDR(r.symbol) + r.addend - refaddr; // 计算相对偏移
    } else if (r.type == R_X86_64_32) {
        // 绝对地址修正
        *refptr = ADDR(r.symbol) + r.addend; // 直接替换为符号地址
    }
}
}

```

以下程序为例

```

int array[2] = {1, 2}; // 全局变量，需绝对地址重定位

int main() {
    int val = sum(array, 2); // sum需PC相对地址重定位
    return val;
}
/*下面是未重定位的汇编*/
0000000000000000 <main>:
    0: 48 83 ec 08          sub $0x8, %rsp
    4: be 02 00 00 00      mov $0x2, %esi
    9: bf 00 00 00 00      mov $0x0, %edi    # &array (需绝对地址重定位)
       a: R_X86_64_32 array # 重定位条目
    e: e8 00 00 00 00      callq 13 <main+0x13> # sum() (需PC相对重定位)
       f: R_X86_64_PC32 sum-0x4 # 重定位条目
   13: 48 83 c4 08          add $0x8, %rsp
   17: c3                   retq

```

绝对地址 array 重定位：修正 `mov $0x0, %edi` 中的 `0x0`，替换为 `array` 的实际地址
定位重定位条目

`r.offset = 0xa` (对应 `mov` 指令中的 `00 00 00 00`)

`r.type = R_X86_64_32` (绝对地址)

`r.symbol = array`，`r.addend = 0`

从符号表可以直接查到符号 `array` 地址：`ADDR(array) = 0x601018`

修正机器码(小端存储)：

原指令：`bf 00 00 00 00` `mov $0x0, %edi`

```
修正后: bf 18 10 60 00      mov $0x601018, %edi
```

相对地址sum重定位: 修正 `callq 13 <main+0x13>` 中的 `00 00 00 00`, 替换为 `sum` 的相对偏移

定位重定位条目

```
r.offset = 0xf (对应 call 指令中的 00 00 00 00)
```

```
r.type = R_X86_64_PC32 (PC相对地址)
```

```
r.symbol = sum, r.addend = -4
```

获取符号和指令地址

```
ADD(s) = ADDR(.text) = 0x4004d0, ADDR(sum) = 0x4004e8
```

```
refaddr = ADD(s) + r.offset = 0x4004d0 + 0xf = 0x4004df
```

```
sum 函数的相对地址 *refptr = (unsigned)(ADD(r.symbol) + r.addend - refaddr) =  
(unsigned) (0x5)
```

修正机器码(小端存储)

```
原指令: e8 00 00 00 00      callq 0x13  
修正后: e8 05 00 00 00      callq 0x4004e8 <sum>
```

最后结果

```
00000000004004d0 <main>:  
4004d0: 48 83 ec 08 sub $0x8,%rsp  
4004d4: be 02 00 00 00 mov $0x2,%esi  
4004d9: bf 18 10 60 00 mov $0x601018,%edi # %edi = &array  
4004de: e8 05 00 00 00 callq 4004e8 <sum> # sum()  
4004e3: 48 83 c4 08 add $0x8,%rsp  
4004e7: c3 retq  
  
00000000004004e8 <sum>:  
4004e8: b8 00 00 00 00 mov $0x0,%eax  
4004ed: ba 00 00 00 00 mov $0x0,%edx  
4004f2: eb 09 jmp 4004fd <sum+0x15>  
4004f4: 48 63 ca movslq %edx,%rcx  
4004f7: 03 04 8f add (%rdi,%rcx,4),%eax  
4004fa: 83 c2 01 add $0x1,%edx  
4004fd: 39 f2 cmp %esi,%edx  
4004ff: 7c f3 jl 4004f4 <sum+0xc>  
400501: f3 c3 repz retq
```


为什么4004f2和4004ff的指令立即数是09和f3? x86跳转指令的偏移量 = 目标地址 - 下一条指令地址。对于4004f2, 下一条指令地址4004f4, 跳转目标地址4004fd, $4004fd - 4004f4 = 0x9$, 所以机器码是eb 09。4004ff同理。

对比可重定位目标文件和可执行目标文件

可重定位目标文件

ELF头
.text节
.rodata节
.data节
.bss节
.symtab节
.rel.txt节
.rel.data节
.debug节
.line
.strtab
节头表

可执行目标文件

ELF 头
段头表(可执行文件)
.init 节
.text节
.rodata节
.data节
.bss节
.symtab
.debug
.line
.strtab
节头表

可执行目标文件与可重定位文件稍有不同：

- ELF头中字段e_entry给出执行程序时 第一条指令的地址，而在可重定位文件中，此字段为0
- 多一个程序头表，也称段头表，是一个结构数组
- 多一个 .init 节，用于定义 _init 函数(用来进行可执行目标文件开始执行时的初始化工作)
- 少两个 .rel 节（无需重定位）

静态链接

链接对象：多个可重定位目标模块（.o 文件）+静态库（标准库、自定义库）（.a 文件，其中包含多个 .o 模块）

静态库

常用的函数打包方法 `.o` 法。但是考虑到目前的链接器框架有两个选择，各有弊端。所以有了传统的解决方案：静态库 `.a` 法

静态库 (`.a` 存档文件)

- 将相关的可重定位目标文件 `.o` 连接到一个带有索引的单个文件 `.a` 中(叫做**静态库**文件/存档文件)
- 链接器通过查找一个或多个存档文件中的符号来解决未解析的外部引用
- 如果存档的一个成员文件 `.o` 解析了符号引用，就把它链接入可执行文件
- 优点：在构建可执行文件时，只需指定库文件名来使用这些在库中定义的函数，链接器只需复制被程序引用的目标模块，减少了可执行文件在磁盘和内存中的大小

常用用户库

`libc.a` C标准库

I/O, 存储器分配,信号处理, 字符串处理, 日期和时间, 随机数, 整数数学运算

`libm.a` C数学库

浮点数学运算(`sin`, `cos`, `tan`, `log`, `exp`, `sqrt` 等)

与静态库链接

编译和创建静态库的命令

```
gcc -c addvec.c mulvec.c          # 编译源文件生成目标文件
ar rcs libvector.a addvec.o mulvec.o # 将目标文件打包成静态库
gcc -o prog1 main2.c ./libvector.a # 链接主程序和静态库生成可执行文件
```

`addvec`执行向量加法，`mulvec`执行向量乘法，就是多个目标文件

使用静态库

链接器的解析外部引用的算法：

- 按照在命令行的顺序扫描`.o`与`.a`文件 § 在扫描期间，保持一个当前未解析的引用列表
- 扫到每一个新的 `.o` 或 `.a` 文件，遇到目标，尝试解析列表中每个未解析的符号引用，而不是在目标中定义的符号。
- 如果在扫描结束时，在未解析符号列表中仍存在任一条目，那么就报错

静态链接器 (`ld`) 的工作方式是**从左到右扫描**，遇到未定义的符号时，会去后面的库中查找。已经扫描过的文件不会再次被检查。因此静态库的**顺序很重要**

参数

-l：让我们手动添加链接库

-L：选择链接库的目录

-I：指定头文件目录

给出命令行，使得静态链接器能够解析所有的符号引用（->是调用关系）

A. `p.o -> libx.a`

B. `p.o -> libx.a -> liby.a`

C. `p.o -> libx.a -> liby.a`且`liby.a->libx.a->p.o`

A. `gcc p.o libx.a` #直接调用

B. `gcc p.o libx.a liby.a`

C. `gcc p.o libx.a liby.a libx.a` #第一次读取目标文件`p.o`时提供了所有符号，但后面的静态库是按需加载，所以`libx.a`再被`liby.a`调用时还要扫描一次，但`p.o`不用

静态库的缺点

- 在保存的可执行文件中存在重复 (每个函数都需要 `libc`)
 - 在运行的多个进程中代码重复复制，浪费内存空间
 - 系统库的更新或者小错误修复要求每个应用程序显式地重新链接，维护复杂
- 解决方案：共享库

共享库(cont.)/动态链接库/DLL

共享库（动态链接库） 是一个 `.so` 的目标模块（`elf` 文件），在运行或加载时，由动态链接器程序加载到任意的内存地址，并和一个和内存中的程序（如当前可执行目标文件）动态链接为一个可执行程序。使用它可节省内存与硬盘空间，方便软件的更新升级。如标准 C 库 `libc.so`

动态链接的方式

第一次加载运行时进行（加载时链接）

- 在 Linux 中，通常由动态链接器（`ld-linux.so`）自动处理
 - 标准C 库 (`libc.so`)通常是动态链接的
 - 软件的部分更新，更轻量便捷
 - 实现方法：应用程序第一次加载和运行时，通过 `ld-linux.so` 动态链接器重定位动态库的代码和数据到某个内存段，再重定位当前应用程序中对共享库定义的符号的引用，然后将控制传递给应用程序（此后共享库位置固定不变）
- 在已经运行后进行（运行时链接）
- 在 Linux 中，通过调用 `dlopen()` 等接口来实现（分发软件包、构建高性能 Web 服务器、库打桩等）
- 库打桩：允许截获对共享库函数的调用，取而代之执行自己的代码。具体分为以下 三种：

1. 编译时:源代码被编译时
 2. 链接时:当可重定位目标文件被静态链接来形成一个可执行目标文件时
 3. 加载/运行时:当一个可执行目标文件被加载到内存中, 动态链接, 然后执行
- 实现方法: 在程序执行过程中, 通过 `dlopen/dlsym` 函数加载和连接共享库, 实现符号重定位, 通过 `dlclose` 卸载动态库。

共享库的例程可以由多个进程共享, 也适合跨语言函数调用, 比如java调用C生成的共享库
共享库的生成

```
gcc -shared -fpic -o libvector.so addvec.c mulvec.c
gcc -o prog2 main.c ./libvector.so
```

- 在任何给定的文件系统中, 对于一个库只有一个 `.so` 文件。所有引用该库的可执行目标文件共享这个文件中的代码和数据, 而不是像静态库的内容那样被复制和嵌入到引用它们的可执行的文件中。在内存中, 一个共享库的 `.text` 节的一个副本可以被不同的正在运行的进程共享。